

Very Brief Machine Learning

Overview

- You may have heard of machine learning/AI/ChatGPT...
- Full subject way too big to tackle here
- We'll restrict ourselves to one example - classifying handwritten digits.
- Dataset we'll use is modified NIST (MNIST). Handwritten digits by high schoolers + NIST employees, with known answers.
- Much of discussion motivated by <http://neuralnetworksanddeeplearning.com/> by Michael Nielsen.

Supervised Learning

- We have a bunch of input images, and we know what the answer should be.
- This is *supervised learning*. c.f. unsupervised learning, where nobody gave you answers.
- We want function that outputs correct digit given input image.
- We want this to work on data *we have not yet seen*. This is not the same as explaining data we already have.

Universal Functions

- Previously, we've had an idea what data should look like. e.g. polynomial - we a) know it's a polynomial, and b) can predict data by evaluating polynomial given coeffs.
- Challenge has been finding m given A such that $d=A(m)$
- Now we have a bunch of input parameters (the images), and the output data (what digit we have). Challenge is to find A .
- *Exceedingly* hard to write down A a priori. What is the function for "Is this an 8? Is this a cat?"?
- One solution - use a universal function for A that can model *any* function given parameters of A .

Neutrons/Perceptrons

- Nature has figured out one way to do this. Neurons get input (either 1 or 0) from other neurons. If sum of inputs greater than a threshold, output 1, otherwise 0.
- Neurons can be more sensitive to some neurons than others. We assign a weight to each connection.
- If we have an input layer of neurons (subscript i) connected to another output layer (subscript j), then we want to know if $\sum W_{ij}x_j > t_i$ where W_{ij} is the weight of the j^{th} input neuron feeding into i^{th} output neuron. x_j is signal from j^{th} neuron, and t_i is threshold for i^{th} neuron to fire.

Bias and Heaviside

- Move threshold to left-hand side and rename bias $b_i = -t_i$.
- Heaviside function $H(x) = 1$ if $x > 0$, 0 if $x < 0$.
- Output of second layer neuron becomes $H(W_{ij}x_j + b_i)$
- We can stack layers to get more complicated behavior (the “deep” in deep learning). Interior layers are called “hidden”
- Binary output neurons are called perceptrons.
- As written, every neuron can speak to every neuron in next layer. Your brain doesn't need this (W is mostly zeros).

Two Examples

- Do we go to the cheese festival?
- Can we make a NAND gate? If yes, we can build arbitrary computer out of perceptrons (since NAND universal)

Sigmoid and Universal

- We're going to fit our parameters ("train the network") by taking gradients. H is bad for training because gradient is either zero or infinity.
- We'll replace H by something that goes smoothly from 0 to 1 around $x=0$. These are called sigmoids ('cause roughly s-shaped).
- Output becomes $S(Wx+b)$, where S is called activation function. One simple is logistic - $\exp(x)/(1+\exp(x))$.
- Cybenko (1989) proved that 2-layer network can approximate any function under some weak constraints on S , as long as S is non-linear.

Cost Functions

- Given a bunch of training data (input to first layer of neurons) and target values (desired output of last layer), and having chosen S , we want to find the “best” W ’s and b ’s
- We need a mathematical definition of “best”. We’ll use familiar squared errors: $\sum(\text{output}-\text{target})^2$. Other choices possible; general term is “Cost Function”. When you hear that, think χ^2 .
- In familiar language, we want to minimize $(d-A(m))^T N^{-1} (d-A(m))$, except (many!) m ’s and d ’s are given, and we want to find A .

Problem Setup

- You might think output should be what digit we have. This is bad.
- What happens if network isn't sure if you have a 3 or 8? Output of 5.5 would not make much sense.
- Instead, have 10 outputs. First output: "is this a 1?", second output "Is this a 2?", with output range 0 (this is not a 2) to 1 (this is definitely a 2).
- Position of largest output is best guess for value.

Setup 2

- We'll have distinct layers of neurons. Layer 0 (the input) feeds into layer 1. Layer 1 feeds into layer 2 etc. This is “feed-forward” network vs. recurrent network where layers feed backwards.
- For MNIST, input layer is 784 (one neuron per pixel). Output layer is 10. We get to decide how many layers, and how many neurons in each - these are hyperparameters.
- Hyperparameters chosen separately from training. We train after choosing hyperparams.
- If x_n is n^{th} neuron layer, $x_{n+1} = S(W_n x_n + b_n)$. Training is finding W_n, b_n .
- Feed-forward easier to fit than recurrent. “Attention is all you need” paper showed how to get benefits of recurrent with feed-forward. Set off LLM explosion.

Backpropagation 1

- We're ready to calculate gradients. For single data, we want dC/dp for parameter p , then we sum over data. Set $N=1$
- Look at *last* layer: $dC/dp = d/dp((y - S(Wx + b))^T (y - S(Wx + b)))$
- Let $r = y - S()$, this is difference between output and truth.
- $dC/db = -2r^T d(S)/db$, $d(S)/db = S'$
- $dC/dW = -2r^T d(S)/dW$, $d(S)/dW = S'()x$
- Also need $dC/dx = -2r^T d(S)/dx$, $d(S)/dx = S'()W$

Backpropagation 2

- How does cost depend on next-to-last layer? Given input x , changing W, b changes output, which is input to the final layer.
- We already know how cost depends on input to final layer, so we can use chain rule to find how cost depends on previous layer.
- Repeat through network, and we have gradient of cost w.r.t each parameter in the network. This is called backpropagation (and was big part of Hinton's Nobel prize).
- Look at `net_example.py`. It calculates gradients, with multiple input data chunks allowed.

SGD

- Now that we have gradients, we want to minimize cost.
- We have a ton of parameters - tens of thousands for MNIST, many billions for the models you've heard of.
- Would be lovely to calculate full curvature, but we can't afford that. Instead, will go downhill.
- "Isn't that bad?" Yeah, but we're kind of stuck. Turns out looping over chunks of data and going a bit downhill works reasonably well.
- This is called stochastic gradient descent (SGD) - we randomly pick chunks ("minibatches") of data, move a little downhill, and repeat.
- "How far downhill should we go?" Great question! That's called the "learning rate" and may need adjustment.

SGD 2

- Case 1: Consider learning rate $\ll$ curvature (i.e. gradient doesn't change much during iteration). $\Delta p \sim \text{sum of gradients}$, so we don't lose (and in fact win by a little) updating during step.
- Case 2: We get much more data. If we have reasonable gradient from data we have, it would be silly to wait until we saw all data to start updating.
- Gradient *extremely* poorly measured in single minibatch, but averaging over batches works. SGD converges much faster in case 2, and even helps in case 1.
- Learning rate scale: $\delta C = \delta p \circ \nabla$, but $\delta p = \alpha \nabla$ and $\delta C \sim C$, so $\alpha \sim C / \nabla^2$. We can use this to get *some* idea of learning rate. In practice very low, but at least gets you started.

Other Minimizers

- Will use SGD for simplicity. Other options usually better.
- “momentum” - if we’re minimizing in a direction, we probably want to remember. $dp_{\text{new}} = a dp_{\text{old}} + (1-a)\text{grad}$. Set a so that descent direction doesn’t wander.
- In deep network, learning unstable since grads tend to grow/shrink exponentially with depth. Can’t get full curvature, but some idea of $\text{grad}(\text{grad})$ useful.
- Methods such as ADAM (one way to combine momentum+moments) useful for training.

Gotta Know When to Fold

- Never forget - we aren't minimizing cost of what we have. We're minimizing cost of what we haven't seen.
- We may have many more params than data points - recipe for overfitting.
- We need to decide when we're done based on something other than minimizing cost of data we are using.

Example - Poly Fitting

- Analagous problem - how many poly terms to use fitting data?
- Before we knew χ^2 . Gave us absolute estimate of when to stop. Now we don't.
- To avoid fitting noise, we could split data in half, fit to one half, and stop when χ^2 for the other half goes up.
- see: `poly_convergence_movie.py`

Training/Validation Data

- We'll do something similar. Use most of data to train - i.e. minimize cost function/fit parameters to training data.
- We'll watch a separate dataset (validation data). When that stops improving, we should stop.
- Crucially, we never fit to validation data.

Let's Fit!

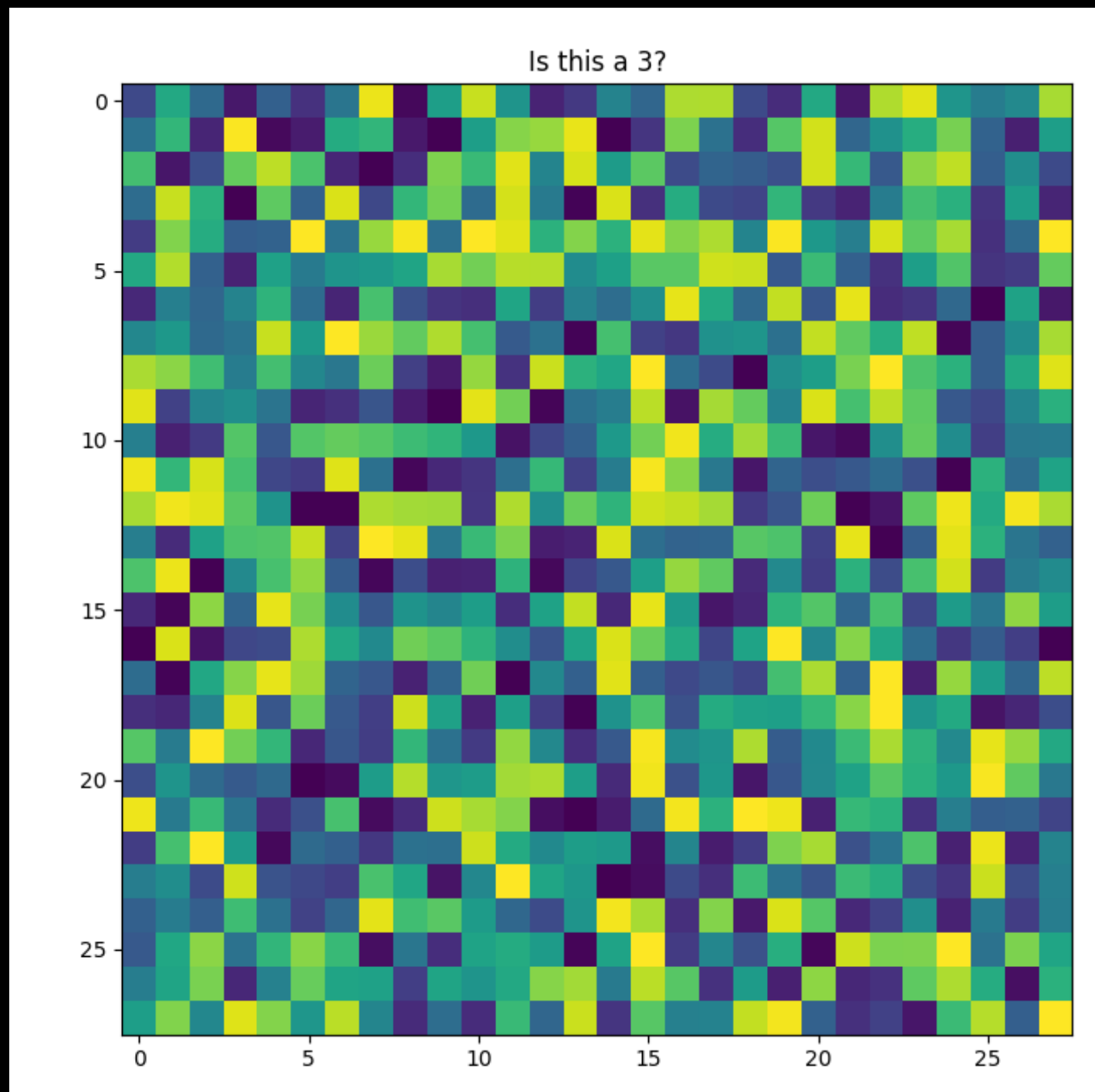
- Look at `net_example.py`
- Will fit to MNIST by minizing cost on training.
- Will report correct fraction on validation.

Hyper Params

- I've tweaked network params (minibatch, depth, #of neurons) to do reasonably well. Typically ~96.5% correct.
- Extensive exploration of hyperparams means we might overfit to validation data.
- Use a third set - the test data - to make sure we haven't overfit.

What Does Network See?

- Possibly sobering exercise - try to create new digits.
- Start with random image, pick target digit, and run backprop.
- Update image with backprop grad of $x[0]$ (keep image bound from 0 to 1)
- Repeat until network thinks images are target digit.
- How does this do? Look at `net_example_reconstruct.py`



Network thinks this is a 3 with 0.945 output

Pruning Network

- Networks can fail spectacularly on data they haven't been trained on ("out of distribution").
- We can improve by only letting network use weights it really wants to.
- Common trick - randomly set some weights to 0 during training.
- Another trick - add prior that weight is 0 to cost function

Summary

- Universal functions very useful for complex problems.
- Nature suggested one class. With tweaks, we can fit.
- Overfitting a big challenge. Split data into parts
- Behavior often poor outside of region where trained.

Challenge

- Train your own network!
- We'll use the digits you've all been writing.
- Submit weights/biases of your favorite network.
- Will give you some of the handwritten numbers to include in training.
- Whoever classifies the non-public remainder gets to drop an extra problem set.